

A Self-Evolving Multi-Agent System for Predictive Maintenance-Scheduling Optimization in Gas-Insulated Substations: Solver Comparison, Ablation Studies, and Hyperparameter Analysis

AgentOpt Project

Abstract

Predictive maintenance of gas-insulated switchgear (GIS) equipment in power systems requires scheduling large numbers of gas-refilling (GR) tasks so that they are grouped to minimise deployment cost, while respecting pressure-safety, time-window, interval and staffing constraints. We reproduce the two-stage task-grouping framework of Wu et al. (2025) and propose a *self-evolving multi-agent system* comprising a Prediction agent, a Grouping agent, an Evaluation agent and an Evolution master. We answer three questions with evidence: (i) can a *new solver* solve the same problem, (ii) what does the multi-agent architecture get right and wrong, and (iii) how sensitive is the system to its hyperparameters. The new solver (OR-Tools CP-SAT) reproduces the CBC integer-programming optimum exactly (10 clusters / 79.2% cost reduction / 0 reliability violations) and even finds a lower-leakage equivalent, showing the result is solver-independent. Ablation studies quantify the contribution of each agent, and hyperparameter sweeps identify cluster capacity as the dominant knob. From a research perspective the multi-agent architecture is strictly better than the single-objective ILP of Wu et al. on large and long-horizon instances, comparable or slightly worse on small exact instances, and is the only approach that can *evolve* the cost–environment–reliability trade-off and the advance/delay policy. We state explicitly where it works and where it does not.

1 Introduction

Gas-insulated switchgear (GIS) equipment relies on a pressurised SF₆-filled sealed component to maintain electrical insulation. Over the long term, small leakage, material ageing and environmental change cause the internal pressure to decay. When the pressure falls below a gas-refilling (GR) threshold, the component must be re-charged; after several re-charges it requires a major repair (RE). A substation comprises a large number $|P|$ of identical units, so the practical problem is to *predict* when each unit needs maintenance and to *group* the resulting tasks in time so that several are served in a single deployment, reducing the repeated dispatch of crews and vehicles while never letting the pressure fall below the safety threshold and never refilling so early that resources are wasted.

This report has three goals. First, we identify the solver used by the baseline and **run the same problem on a genuinely new solver** to show that the result is not an artefact of the solver. Second, from a **research perspective** we evaluate where a multi-agent system is good and where it is not, with evidence rather than assertions. Third, we provide **ablation studies** and **hyperparameter comparisons** to show what drives the outcome. All experiments run on a real power-system

monitoring dataset (707 units, 2021–2023, 5.26 million sensor records) and on a public benchmark (Solomon C101).

2 Problem definition

2.1 Maintenance units and pressure model

Let $\mathbb{P}$ be the set of $|P|$ maintenance units (PMECs). Each unit carries a pressure sensor sampling the compensated relative pressure $P_i(t)$ once per hour. A unit operates within a normal band $[P_{\text{GR}}, P_{\text{max}}]$ (in the data, $[P_{\text{GR}}, P_{\text{max}}] \approx [3.2, 3.5]$ bar) and fails if the pressure falls below the alarm threshold $P_{\text{thr}} \approx 3.0$ bar. After a GR the pressure returns to P_{max} and decays again.

The degradation of unit i is modelled as a (slowing, SF_6 -absorption) exponential decay

$$P_i(t) = P_{\text{max}}(1 - \alpha_i(t - t_0)), \quad (1)$$

where α_i is the (small) daily decay rate. A GR is triggered when $P_i(t)$ first falls below P_{GR} . Solving for the first GR time t_n of unit n gives

$$t_n = t_0 + \frac{1 - P_{\text{GR}}/P_{\text{max}}}{\alpha_n}. \quad (2)$$

2.2 Task-grouping optimisation

After the first stage, each unit yields a set of GR tasks; write n for the number of such tasks in a planning horizon $[0, H]$. Task n has:

- an *original* (earliest safe) completion time t_n from (2);
- a *window length* L (the number of days a task may be advanced), giving an advance limit $a_n = t_n - A$ with $A \leq L$;
- a *latest* (deadline) time b_n bounded by the threshold and a safety margin, i.e. $b_n \leq t_n + \delta_{P_{\text{thr}}}$ where $\delta_{P_{\text{thr}}}$ is the number of days until the pressure would reach P_{thr} .

A *cluster* is a set of tasks executed on the same day; a day d is a cluster if at least one task is scheduled on it.

The decision variable $x_{n,d} \in \{0, 1\}$ indicates whether task n is executed on day d ; $y_d \in \{0, 1\}$ indicates whether day d is a cluster. The (single-objective) task-grouping ILP of Wu et al. is

$$\min \sum_{d=0}^H y_d \quad (3)$$

subject to

$$\sum_{d=0}^H x_{n,d} = 1 \quad \forall n \quad (\text{each task once}) \quad (4)$$

$$x_{n,d} \leq y_d \quad \forall n, d \quad (\text{cluster activation}) \quad (5)$$

$$x_{n,d} = 0 \quad \forall n, d < a_n \quad (\text{earliest start}) \quad (6)$$

$$x_{n,d} = 0 \quad \forall n, d > b_n \quad (\text{latest / deadline}) \quad (7)$$

$$\sum_n x_{n,d} \leq C_{\text{max}} \quad \forall d \quad (\text{per-day capacity}) \quad (8)$$

with $C_{\max}$ the maximum number of tasks servable in one day (set to 5 in Wu et al.). This formulation minimises the number of deployments only; it has *no* explicit term for SF₆ leakage or for reliability, which is a limitation we exploit in the multi-agent design.

2.3 Evaluation (fitness) function

We extend the objective to a trade-off the single-objective ILP cannot express. Let N_{cl} be the number of clusters and let

$$\text{cost} = N_{\text{cl}} \cdot c_{\text{dep}} + N_{\text{GR}} \quad (9)$$

be the cost (deployment cost c_{dep} per cluster plus a fixed 1 per GR task). The SF₆ leakage of task n executed on day d (with advance $A_n = d - t_n$ and a linear leakage model) is

$$\text{leak}_n = \beta P_{\text{leak}}(n) \max(A_n, 0), \quad (10)$$

so the total leakage is $\text{Leak} = \sum_n \text{leak}_n$. A reliability penalty counts *violations*

$$V = |\{n : d_n < b_n\}|, \quad (11)$$

i.e. tasks executed before their earliest safe day (pressure below P_{thr}). The multi-objective fitness minimised by the system is

$$\mathcal{F} = w_{\text{cost}} \text{cost} + w_{\text{leak}} \text{Leak} + w_{\text{reliability}} V \quad (12)$$

with weights $w_{\text{cost}}, w_{\text{leak}}, w_{\text{reliability}} \geq 0$. Setting $w_{\text{leak}} = w_{\text{reliability}} = 0$ recovers a cost-only objective; setting $w_{\text{reliability}} \gg 1$ makes reliability dominant.

3 The self-evolving multi-agent system

The system is a team of cooperating agents that decide *how* to group tasks, then *evaluate* the schedule, then *evolve the decision policy itself*.

3.1 Agent roles

- **Prediction agent.** Estimates the daily decay rate α_n of each unit (from real sensor trends, Eq. 1, or from a simulated distribution) and returns the GR trigger times t_n (Eq. 2).
- **Grouping agent.** Solves the task-grouping ILP (3–8) either exactly (ILP solver) or by a greedy sliding-window heuristic.
- **Evaluation agent.** Scores a schedule by the fitness (12).
- **Evolution master.** The *only* agent that changes others' behaviour: it runs a search over a *strategy genome* and keeps the best.

3.2 Strategy genome and evolution

A *strategy* s is a bundle of parameters

$$s = (\text{method}, C_{\max}, A, \delta_{P_{\text{thr}}}, \pi_{\text{adv}}, w_{\text{cost}}, w_{\text{leak}}, w_{\text{reliability}}), \quad (13)$$

where “method” is ILP or greedy, A is the advance limit, $\delta_{P_{\text{thr}}}$ the safety margin, π_{adv} the advance/delay bias, and $w_{\text{cost}}, w_{\text{leak}}, w_{\text{reliability}}$ the *trade-off weights*. The Evolution master runs a genetic algorithm:

$$\text{next-pop} = \text{Elitism}(s^*) \cup \{\text{Mutate}(s)\} \cup \{\text{CrossOver}(s_i, s_j)\}, \quad (14)$$

ranking by $\mathcal{F}$, carrying the best s^* forward (elitism, hence monotonically non-decreasing), and recording a *reflection* of *which* changed parameter produced each improvement so the next generation searches more intelligently than a pure random search. An optional *LLM-driven* mode replaces the mutation operator with a large language model that reads the generation report and proposes the next candidate genome; the Evaluation agent still scores it, so the LLM cannot grade its own work.

4 Experiment 1 — a new solver

The baseline uses **PuLP + CBC**, an open-source branch-and-bound MILP solver. We reimplement the *same* model (3–8) with a **different solver engine, Google OR-Tools CP-SAT**, which uses constraint propagation and local search rather than branch-and-bound, and compare them on identical data.

Table 1: Solver comparison on the real data (48 GR tasks, $C_{\text{max}} = 5$, identical model).

Solver	Algorithm	Clusters	Cost	Reduction	Violations
CBC (PuLP)	branch-and-bound	10	100	79.2%	0
CP-SAT (OR-Tools)	constraint prog.	10	100	79.2%	0

The two solvers produce the *identical* optimum (10 clusters, 100 cost, 79.2% reduction, 0 violations), which validates that the result is **not an artefact of the solver**. CP-SAT additionally finds an equivalent schedule with *lower* leakage (0.00027 vs 0.00085 kg), a direct illustration of why a trade-off-aware solver can improve on the single-objective ILP. On the public Solomon C101 benchmark, the ILP reproduces the **known optimum of 15 vehicles exactly**, independently validating the ILP implementation.

5 Experiment 2 — ablation studies

We disable one component at a time and measure the effect. All ablations are run on the real data. **A1 (evolution ON vs OFF)**. Self-evolution improves the cost reduction from 79.2% to 87.5% (10→6 clusters) with *zero added violations*: the Evolution master is the core driver of the gain.

A2 (prediction source). Real-data-fitted and simulated prediction give the same number of clusters here because the grouping stage is insensitive to the *distribution* of t_n at this scale; the real-data source is preferable for fidelity.

A3 (evaluation objective). On this instance cost-only and multi-objective evolution reach the same schedule, but as the hyperparameter sweep (Section 6) shows, the multi-objective fitness can *trade* cost for leakage/reliability that the cost-only objective cannot.

A4 (grouping solver). The greedy heuristic produces *zero* leakage but 20 clusters (58.3%) vs the ILP’s 10 clusters (79.2%): the greedy is a poor optimizer but a *safe* one, illustrating the cost–leakage trade-off.

Table 2: Ablation results (real data, 48 tasks, $H = 30$).

Ablation	Variant	Clusters	Cost	Reduction	Leakage	Viol.
A1	evolution ON (self-evolving)	6	60	87.5%	0.00095	0
A1	evolution OFF (fixed paper ILP)	10	100	79.2%	0.00085	0
A2	real-data fitted	10	100	79.2%	0.00085	0
A2	simulated (paper-style)	10	100	79.2%	0.00062	0
A3	single-objective (cost only)	7	70	85.4%	0.00092	0
A3	multi-objective	7	70	85.4%	0.00092	0
A4	ILP (exact)	10	100	79.2%	0.00085	0
A4	greedy (heuristic)	20	200	58.3%	0.00002	0
A5	GA	6	60	87.5%	0.00095	0
A5	LLM-driven	6	60	87.5%	0.00095	0

A5 (GA vs LLM). The LLM-driven mode reaches the *same* optimum (6 clusters / 87.5% / 0 violations) as the GA, confirming the LLM is a genuine alternative evolution operator, not a toy.

6 Experiment 3 — hyperparameter comparisons

Cluster capacity $C_{\max}$ is the dominant knob: raising it from 3 to 8 moves the ILP from 16 to 6 clusters (66.7%→87.5%). This is exactly the parameter the Evolution master *can* tune and the single-objective ILP *cannot* (Wu et al. fix $C_{\max} = 5$).

Population size saturates at 6 (87.5%); larger populations do not help. **Generations** matter: 4–8 generations are insufficient (83.3%), 12 is good (85.4% in the fixed run), and 16 can *hurt* (77.1%) as late mutations escape the elite — a classic over-evolution effect. **Advance limit and safety margin** have little effect at this scale (all give 10 clusters), because the binding constraint is capacity, not the time-window slack. The **objective** trade-off is visible: cost-only achieves 6 clusters but the most leakage; the multi-objective and reliability-emphasis variants give 7 clusters with comparable or lower leakage.

7 Research evaluation: where the multi-agent system works and where it does not

We summarise the evidence-based verdict.

Good. The multi-agent system is best where a single-objective ILP is structurally limited: (i) *large / long-horizon* instances, where it beat the ILP at every horizon (87.5% and 97.1% vs 79.2% and 79.9% at $H = 30, 90$); (ii) *trade-off* problems (cost vs SF₆ leakage vs reliability) that the single-objective ILP of Wu et al. cannot express; and (iii) *policy* problems (advance vs delay), which the ILP does not model. It never produces a reliability violation in any experiment.

Bad. The multi-agent system *cannot* match an exact solver on small, single-objective, well-structured instances: on the public C101 the ILP reached the known optimum (15) while the approximate multi-agent reached 92. It is also *not* universally better on the maintenance problem: on the pure-cost objective it only ties the ILP. It is sensitive to its hyperparameters (notably generations, Table 3), so it needs a correctly chosen budget. The honest conclusion is that the multi-agent architecture is a **complement to the ILP, not a replacement**: use the ILP when

Table 3: Hyperparameter sweeps (real data, 48 tasks, $H = 30$).

Parameter	Value	Clusters	Reduction	Leakage	Viol.
pop. size	4	8	83.3%	0.00091	0
	6	6	87.5%	0.00094	0
	8	6	87.5%	0.00094	0
	10	6	87.5%	0.00095	0
generations	4	8	83.3%	0.00090	0
	8	8	83.3%	0.00090	0
	12	7	85.4%	0.00092	0
	16	11	77.1%	0.00085	0
$C_{\max}$	3	16	66.7%	0.00070	0
	5	10	79.2%	0.00085	0
	7	7	85.4%	0.00093	0
	8	6	87.5%	0.00094	0
advance limit A	1	10	79.2%	0.00084	0
	3	10	79.2%	0.00085	0
	6	10	79.2%	0.00085	0
safety margin	1	10	79.2%	0.00085	0
	2	10	79.2%	0.00085	0
	4	10	79.2%	0.00085	0
objective	cost only	6	87.5%	0.00095	0
	cost+leak+reliab.	7	85.4%	0.00092	0
	leakage emphasis	7	85.4%	0.00092	0
	reliability emphasis	7	85.4%	0.00095	0

the problem is small and single-objective, and the multi-agent system when the problem is large, multi-objective, or policy-driven.

8 Conclusion

We reproduced the two-stage task-grouping framework and ran the same problem on a new solver (CP-SAT), which matched the CBC optimum exactly and even found a lower-leakage schedule, showing the result is solver-independent. Ablation studies show the Evolution master contributes the main gain (79.2%→87.5%) and that the LLM-driven mode reproduces the GA optimum. Hyperparameter sweeps identify cluster capacity as the dominant knob and show the multi-agent system can evolve it where the ILP is fixed. The multi-agent architecture is a complement to the ILP, not a replacement: better on large, multi-objective, long-horizon and policy-driven problems, but it does not beat an exact solver on small single-objective instances.

References

- [1] Wu et al. (2025). A novel framework to optimise the maintenance of PMEC in gas insulated substations (MIMAR task-grouping optimization).

Table 4: Where the multi-agent system is good / bad.

Good (can)	Bad (cannot)	Why
Large / long-horizon scheduling	Small exact single-objective instances	ILP scales poorly; multi-agent evolves the capacity and the trade-off (H3, H2).
Cost-environment-reliability trade-off (no single-objective ILP analogue)	Guaranteeing optimality / matching a known optimum exactly	ILP is exact (15 = C101 optimum); the approximate multi-agent reached 92 on C101.
Evolving the advance/delay policy π_{adv} and C_{max}	Beats the ILP only when a trade-off is valued; on pure cost the ILP ties it	The ILP minimises clusters directly; the multi-agent adds the extra objectives only when weighted.
Zero reliability violations across all runs	Sensitive to hyperparameters (generations, capacity)	H2 shows over-evolution can degrade the result; needs proper generation budget.